

Chapter 05

Output Parsers

Forcing structure onto raw LLM text — String, JSON, Structured, and Pydantic parsers, and when to reach for each one.

WHAT THIS CHAPTER COVERS

- ▶ Why raw LLM output cannot be sent to other systems
- ▶ What an Output Parser is and does
- ▶ String Output Parser and the chain benefit
- ▶ JSON Output Parser and its schema limitation
- ▶ Structured Output Parser (legacy, brief overview)
- ▶ Pydantic Output Parser and data validation
- ▶ `partial_variables` and `format_instructions`
- ▶ Choosing the right parser for a use case

Output Parsers

RECAP — CHAPTER 4

Chapter 4 covered **Structured Output**: forcing an LLM to respond in a defined schema instead of free text. It introduced two categories of LLMs — those that natively *can* return structured output (fine-tuned for it) and those that *cannot*, and showed the `with_structured_output` function for working with the first category using GPT-style models. This chapter deals with the second category: models that cannot natively give structured output, and the tool used to force structure onto them — **Output Parsers**.

1. What Are Output Parsers

Definition

Output Parsers in LangChain are classes that help convert raw, textual LLM responses into structured formats such as JSON, CSV, or Pydantic models. They ensure consistency, validation, and ease of use when passing LLM output into other applications.

Why It Exists

An LLM, by default, always replies with unstructured text. Unstructured text cannot be handed off to another system — a database, an API, or a downstream function — because those systems expect a predictable shape of data. Structured output (introduced in the previous chapter) solves this by forcing the LLM to respond in a defined schema like JSON, which downstream systems can consume directly.

Some LLMs are fine-tuned to natively return structured output when asked — these are the *can*-category models. Most open-source models are not fine-tuned this well and struggle to reliably return structured output on their own — these are the *cannot*-category models. Output Parsers solve the problem for the second category, but they are equally usable with the first category too.

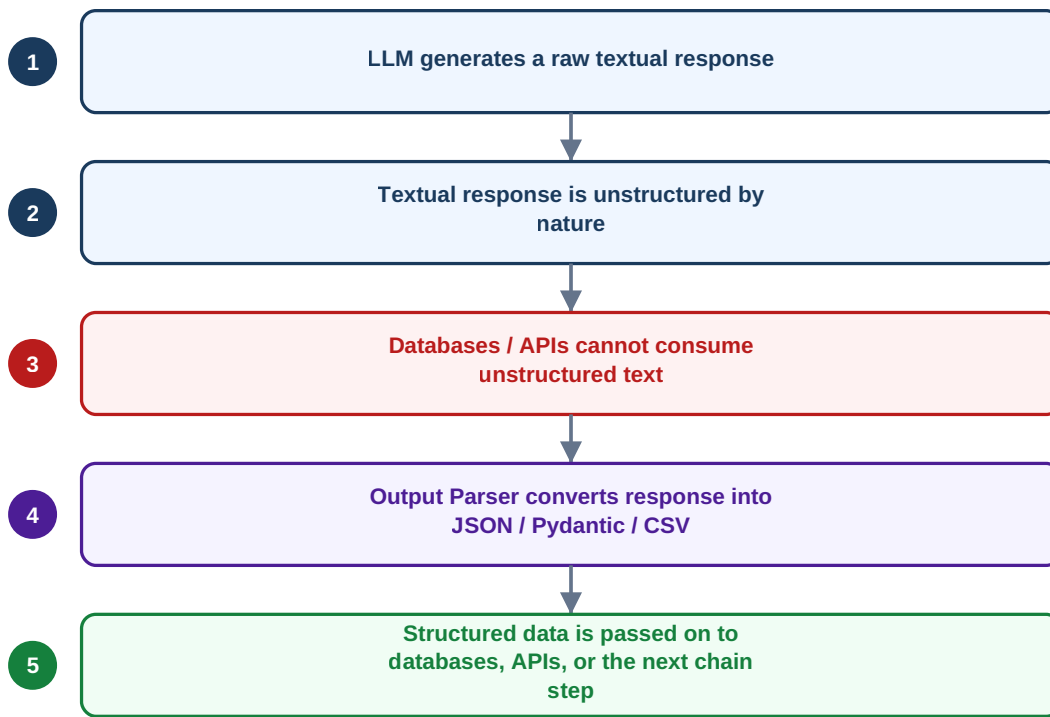


Figure 5.1 — Why unstructured LLM text needs an Output Parser before it can reach another system.

IMPORTANT

Output Parsers are not limited to models that lack native structured-output support. They work seamlessly with *both* categories of models — the ones that natively support structured output and the ones that don't.

The Four Most-Used Output Parsers

LangChain ships many output parsers (CSV, list, datetime, enum, output-fixing, and more), but four of them cover the overwhelming majority of real use cases:

1. **String Output Parser** — extracts plain text from the response.
2. **JSON Output Parser** — forces the LLM to reply in JSON format.
3. **Structured Output Parser** — enforces a predefined field schema (now a legacy/deprecated approach).
4. **Pydantic Output Parser** — enforces a schema and validates the data types of the response.

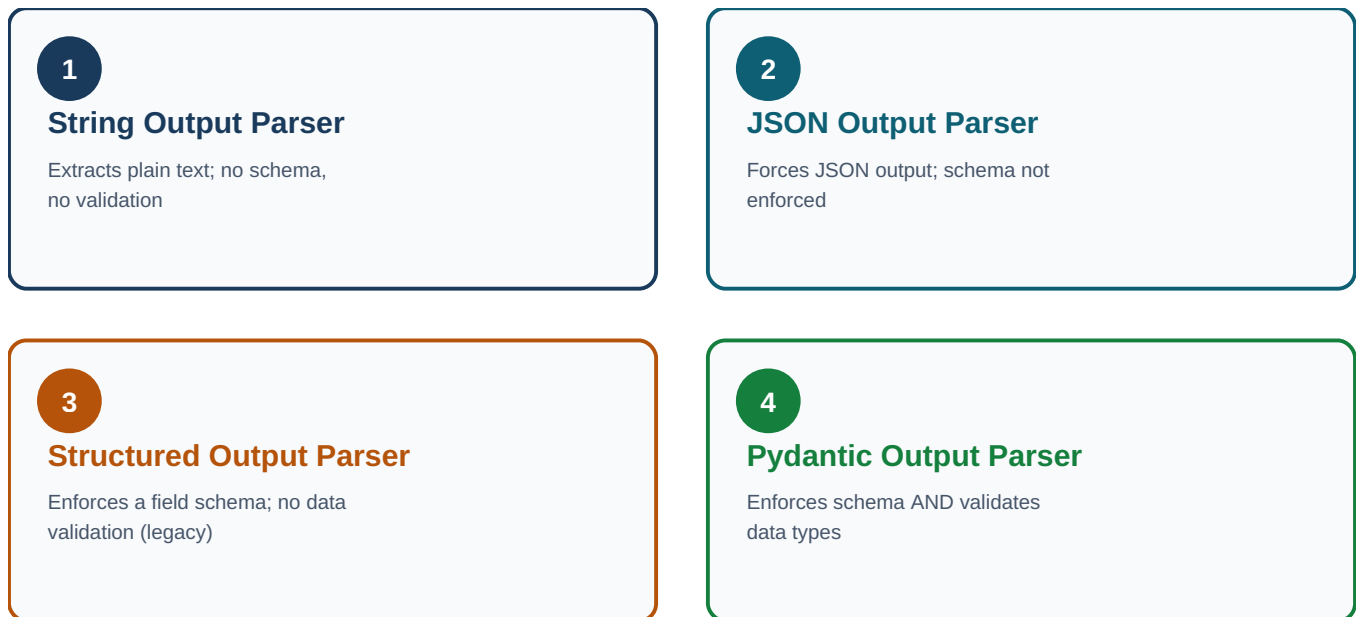


Figure 5.2 — The four most commonly used Output Parsers in LangChain, in increasing order of strictness.

Quick Revision — What Are Output Parsers

- Output Parsers convert raw LLM text into structured formats like JSON, CSV, or Pydantic models.
- LLMs always return unstructured text by default; other systems (databases, APIs) cannot consume this directly.
- Some LLMs natively support structured output ("can" models); most open-source models do not ("cannot" models).
- Output Parsers work with both categories of models, not just the ones lacking native support.
- The four most common parsers are: String, JSON, Structured, and Pydantic Output Parser.

2. String Output Parser

Definition

The **String Output Parser** is the simplest output parser in LangChain. Its only job is to take the LLM's response and convert it directly into a plain string.

Why It Exists

Normally, a chat model's response is not just text — it also carries metadata (token usage, completion tokens, model info, and so on). To get the actual text, developers must always access `result.content`. The String Output Parser removes this repetitive step by handing back the plain text directly, which becomes especially useful when a response has to be fed into a *second* prompt as part of a chain.

How It Works

Consider a two-step use case: first ask the LLM to write a detailed report on a topic, then feed that report back into the same LLM asking for a 5-line summary.

WITHOUT STRING OUTPUT PARSER

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct",
    task = "text-generation"
)

model = ChatHuggingFace(llm = llm)

# 1st Prompt - Detailed Report
template1 = PromptTemplate(
    template = "Write a detailed report on {topic}.",
    input_variables = ["topic"]
)

# 2nd Prompt - Summary
template2 = PromptTemplate(
    template = "Write the summary on the following text \n {text} \n in 5 lines.",
    input_variables = ["text"]
)

prompt1 = template1.invoke({"topic": "Black Holes"})
result1 = model.invoke(prompt1)

# Second prompt manually reads result1.content
prompt2 = template2.invoke({"text": result1.content})
result2 = model.invoke(prompt2)

print(result2.content)
```

Here, `result1.content` must be manually extracted and passed into the second prompt. Every hop between prompt and model requires manually pulling out `.content`.

WITH STRING OUTPUT PARSER (USING A CHAIN)

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct",
    task = "text-generation"
)

model = ChatHuggingFace(llm = llm)

template1 = PromptTemplate(
    template = "Write a detailed report on {topic}.",
    input_variables = ["topic"]
)
template2 = PromptTemplate(
    template = "Write the summary on the following text /n {text} \n in 5 lines.",
    input_variables = ["text"]
)

# Creating Output Parser
parser = StrOutputParser()

# Forming Chain: template1 -> model -> parser -> template2 -> model -> parser
chain = template1 | model | parser | template2 | model | parser

result = chain.invoke({"topic": "Black Holes"})

print(result)
```

The parser is inserted directly inside the pipe (`|`) chain between the model and the next prompt template. Once the chain is built, calling `chain.invoke(...)` runs the entire pipeline automatically — the parser extracts the plain text at each step behind the scenes, so no manual `.content` access is needed anywhere in the flow.

BEST PRACTICE

The most common use of the String Output Parser is inside multi-step **Chains**. Whenever an LLM's plain-text output must flow into another prompt template, wiring the parser directly into the chain (`template | model | parser`) is far cleaner than manually extracting `result.content` at every step.

COMMON MISTAKE

When invoking a chain or template that has no defined `input_variables`, an empty dictionary must still be passed to `.invoke()` — e.g. `chain.invoke({})`. Omitting the argument entirely raises a *missing positional argument* error.

🔄 Quick Revision — String Output Parser

- String Output Parser converts an LLM's raw response object into a plain string.
- It removes the need to repeatedly call `result.content` when passing data between prompts.
- It is most useful inside multi-step Chains, e.g. `template1 | model | parser | template2 | model | parser`.
- Even with no input variables, `.invoke()` must receive an empty dictionary `{}`.
- This same code works with any chat model — Hugging Face, OpenAI, Groq, Gemini, or a local model.

3. JSON Output Parser

Definition

The **JSON Output Parser** forces an LLM to return its output strictly in JSON format. It is the quickest possible way to get JSON output from an LLM.

How It Works

The parser exposes a `get_format_instructions()` method. This method returns an instruction string (e.g. "Return a JSON object") that gets inserted into the prompt via a **partial variable**. A partial variable is filled in *before* runtime by a function call, rather than being supplied by the user at invoke time — which is exactly why it is called "partial" rather than a normal input variable.

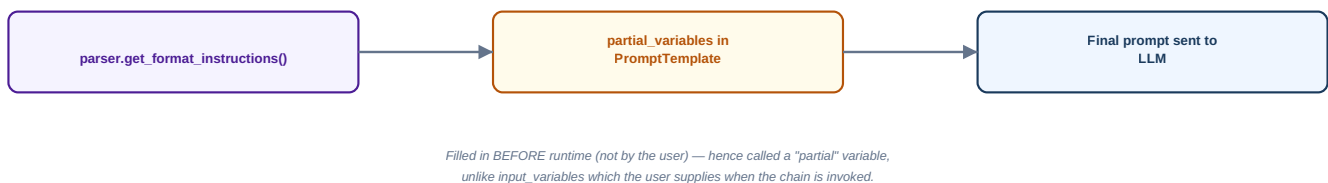


Figure 5.3 — How `format_instructions` is injected into the prompt as a partial variable before the chain runs.

```

from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import JsonOutputParser
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct",
    task = "text-generation"
)

model = ChatHuggingFace(llm = llm)

parser = JsonOutputParser()

template = PromptTemplate(
    template = "Give me the name, age and city of a fictional character \n
{format_instruction}",
    input_variables = [],
    # partial_variables are filled BEFORE runtime, not by the user at invoke time
    partial_variables = {"format_instruction":parser.get_format_instructions()}
)

# Using Chain
chain = template | model | parser

result = chain.invoke({})

print(result)
print(type(result))

```

Once the chain (`template | model | parser`) is invoked, the flow is: the template builds the full prompt (including the injected format instructions) → the model generates a raw response → the parser calls `.parse()` internally on `result.content` and returns a Python dictionary, since Python treats a JSON object as a dictionary.

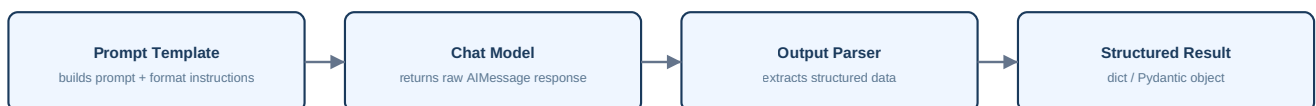


Figure 5.4 — The general prompt → model → parser pipeline shared by all four parsers.

COMMON MISTAKE

The JSON Output Parser has no way to enforce a specific schema. For example, asking for "5 facts about a topic" may return a single key with a list of 5 facts, when the desired structure was actually `fact_1`, `fact_2`, `fact_3` as separate keys. The instructions in the prompt can hint at a structure, but there is no guarantee the LLM will follow it exactly.

IMPORTANT

The biggest limitation of the JSON Output Parser is that it **does not enforce a schema**. It guarantees valid JSON syntax, but not any particular field structure. Enforcing a schema requires the Structured Output Parser or the Pydantic Output Parser instead.

🔄 Quick Revision — JSON Output Parser

- JSON Output Parser forces the LLM to reply in valid JSON format.
- `get_format_instructions()` generates the instruction text injected into the prompt.
- This instruction text is inserted via a partial variable, filled before runtime rather than by the user.
- The parsed result is returned as a Python dictionary.
- Its biggest limitation is that it cannot enforce a specific field schema.

4. Structured Output Parser (Legacy Overview)

Definition

The **Structured Output Parser** is an output parser that extracts structured JSON data from LLM responses based on a predefined set of field schemas, built using the `ResponseSchema` class.

REAL WORLD INSIGHT

Deprecation note: the Structured Output Parser is no longer actively maintained or recommended in current LangChain usage. It is covered here only briefly, to build a conceptual bridge between the JSON Output Parser (no schema enforcement) and the Pydantic Output Parser (schema enforcement + data validation). New projects should prefer the Pydantic Output Parser instead.

How It Works (Conceptually)

Instead of only asking for JSON, a list of `ResponseSchema` objects is defined upfront — each with a **name** and a **description** — and the parser is built from that list using `StructuredOutputParser.from_response_schemas(...)`. This lets the LLM know exactly which keys are expected in its JSON response, unlike the plain JSON Output Parser where the structure is left entirely to the LLM's discretion.

```
from langchain.output_parsers import StructuredOutputParser, ResponseSchema

response_schemas = [
    ResponseSchema(name="fact_1", description="Fact 1 about the topic"),
    ResponseSchema(name="fact_2", description="Fact 2 about the topic"),
    ResponseSchema(name="fact_3", description="Fact 3 about the topic"),
]

parser = StructuredOutputParser.from_response_schemas(response_schemas)
```

Notably, this parser is imported from the broader `langchain` package rather than `langchain_core` — `langchain_core` holds only the most reusable, high-frequency components, and the Structured Output Parser was not considered important/reusable enough to sit there, unlike the JSON Output Parser.

Limitation

Its one meaningful advantage over the JSON Output Parser is schema enforcement — it guarantees which *keys* appear in the response. However, it still cannot perform **data validation**. If a schema expects `age` as an integer but the LLM returns the string `"35 years"`, the parser has no mechanism to catch or reject this. This exact gap is what motivates the Pydantic Output Parser, covered next.

🔄 Quick Revision — Structured Output Parser

- Structured Output Parser enforces a defined set of field names via `ResponseSchema` objects.
- It is imported from the broader `langchain` package, not `langchain_core`.
- It solves the JSON Output Parser's lack of schema enforcement.
- It still cannot validate data types — e.g. it cannot catch an `age` field returned as a string.
- This parser is now considered legacy/deprecated in current LangChain usage — prefer the Pydantic Output Parser.

5. Pydantic Output Parser

Definition

The **Pydantic Output Parser** is a structured output parser that uses Pydantic models to enforce both schema structure and data-type validation when processing LLM responses.

Why It Exists

The Structured Output Parser can enforce a schema's keys, but not the data types of those keys. The Pydantic Output Parser closes this gap: because the schema is defined as a Pydantic `BaseModel`, LangChain gets Pydantic's validation engine for free — type mismatches, constraint violations (like `gt=18`), and malformed values can all be caught automatically.

CORE FEATURES

- **Strict Schema Enforcement** — exact data types and constraints are specified and enforced.
- **Type Safety** — if the LLM returns a slightly incorrect type, type coercion/validation rules apply.
- **Easy Validation** — validation is handled by Pydantic itself, no custom checking code required.
- **Seamless Integration** — works cleanly with the rest of the LangChain component ecosystem (prompts, chains, models).

How It Works

A `Person` Pydantic model is defined with three fields — `name` (string), `age` (integer, constrained to be greater than 18), and `city` (string) — each documented with a `Field(description=...)`. This Pydantic class is passed into `PydanticOutputParser(pydantic_object=Person)` to build the parser, and the rest of the flow mirrors the JSON Output Parser exactly: `get_format_instructions()` is injected as a partial variable, and the chain is `template | model | parser`.

```

from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct",
    task = "text-generation"
)

model = ChatHuggingFace(llm = llm)

# Pydantic object acts as the schema AND the validator
class Person(BaseModel):

    name : str = Field(description = "Name of the person.")
    age : int = Field(gt=18, description = "Age of the person.")
    city : str = Field(description = "Name of the city Person belongs to.")

parser = PydanticOutputParser(pydantic_object = Person)

template = PromptTemplate(
    template = "Generate the name, age and city of a fiction {place} character \n
{format_instructions}",
    input_variables = ["place"],
    partial_variables = {"format_instructions" : parser.get_format_instructions()}
)

# Using Chain
chain = template | model | parser

result = chain.invoke({"place":"India"})

print(result)

```

Internally, when this chain runs, the format instructions actually sent to the LLM describe the exact JSON schema expected (including field names, types, and constraints), instructing it to conform to that schema as an example. The final parsed **result** is a validated **Person** object rather than a plain dictionary.

INTERVIEW TIP

Be ready to explain the progressive improvement across the four parsers: String (plain text only) → JSON (structure, but no schema) → Structured (schema, but no validation) → Pydantic (schema + validation). This progression is a very common interview framing for output parsers.

BEST PRACTICE

When the downstream system depends on strict data types (e.g. an **age** field that absolutely must be an integer, or a **price** that must be a float), always prefer the Pydantic Output Parser over the JSON or Structured Output Parser, since only Pydantic validation can guarantee type correctness.

Quick Revision — Pydantic Output Parser

- Pydantic Output Parser enforces both schema structure and data-type validation.
- The schema is defined as a Pydantic BaseModel class instead of a list of ResponseSchema objects.
- Field() can attach descriptions and constraints (e.g. gt=18) to each attribute.
- get_format_instructions() and partial_variables work the same way as with the JSON Output Parser.
- The final parsed result is a validated Pydantic object, not a plain dictionary.
- This is the most robust of the four parsers, combining structure with validation.

K. Key Takeaways

- `JsonOutputParser` convert raw, unstructured LLM text responses into structured formats (JSON, Pydantic objects) that other systems can consume.
- `TextOutputParser` simply extracts plain text, removing the need to manually access `result.content`, and shines inside multi-step Chains.
- `JsonKeyOutputParser` forces valid JSON output but cannot enforce which keys or structure that JSON must have.
- `ResponseSchemaOutputParser` (legacy) enforces a specific key schema via `ResponseSchema`, but still cannot validate data types.
- `PydanticOutputParser` enforces both schema and data-type validation by defining the schema as a Pydantic `BaseModel`.
- `FormatPromptInstructions` inject format instructions into a prompt before runtime, distinct from user-supplied `input_variables`.
- All four parsers integrate identically into a LangChain pipeline as `template | model | parser`, regardless of the underlying LLM provider.

R. Revision Sheet

Output Parser

A LangChain class that converts an LLM's raw textual response into a structured format such as JSON, CSV, or a Pydantic object.

String Output Parser

Extracts plain text from a response, avoiding repeated `.content` access; most useful when chaining multiple prompt/model steps together.

JSON Output Parser

Forces the LLM to return valid JSON; does not enforce any particular key structure or schema.

Structured Output Parser

Legacy parser that enforces a defined field schema via `ResponseSchema`, but cannot validate data types. No longer actively recommended.

Pydantic Output Parser

Enforces both schema and data-type validation by defining the expected output as a Pydantic `BaseModel` class.

`partial_variables`

Prompt template variables filled in before runtime (e.g. via `get_format_instructions()`), as opposed to `input_variables` supplied by the user at invoke time.

I. Interview Questions

Q1. What problem do Output Parsers solve in LangChain, and why can't raw LLM output be sent directly to a database or API?

Q2. Explain the difference between models that "can" and "cannot" natively produce structured output, and how Output Parsers relate to both categories.

Q3. What is the main benefit of using the String Output Parser inside a Chain, compared to manually accessing `result.content`?

Q4. What is a partial variable in a PromptTemplate, and how does it differ from a regular input variable?

Q5. What is the biggest limitation of the JSON Output Parser, and how does the Structured Output Parser attempt to address it?

Q6. Why is the Structured Output Parser considered a legacy approach today, and what parser has effectively replaced it?

Q7. How does the Pydantic Output Parser improve on the Structured Output Parser?

Q8. Walk through what happens internally when a chain built as `template | model | parser` is invoked.

Q9. Why is `StructuredOutputParser` imported from `langchain` instead of `langchain_core`, unlike the JSON and Pydantic parsers?

Q10. Give an example of a real-world scenario where data-type validation (Pydantic) would matter more than simple schema enforcement (Structured Output Parser).

F. Further Reading

- LangChain Output Parsers documentation (official docs) — overview of all available parser types.
- Pydantic documentation — `BaseModel`, `Field`, and validation constraints.
- LangChain source: `langchain_core.output_parsers` and `langchain.output_parsers` modules.
- Additional parser types worth exploring: CSV Output Parser, List Output Parser, Datetime Output Parser, Enum Output Parser, and the Output-Fixing Parser (for auto-correcting malformed responses).